

```

# NOAA OISST NetCDF Environmental Stress Analysis
# Google Colab Script Version
# Author: Beija Richardson

# =====
# PURPOSE
# =====
# This Colab script uses the real NOAA OISST NetCDF files uploaded for the project.
# - loading NOAA OISST .nc files
# - extracting SST anomaly values from the North Atlantic region
# - creating a daily regional average anomaly dataset
# - engineering rolling temperature, temperature change, volatility, and ESMS
# - creating summary statistics, correlation tables, and figures
# - saving processed outputs for the final report and GitHub

# =====

!pip install xarray netCDF4 -q

from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import xarray as xr
from pathlib import Path

# Example files:
# oisst-avhrr-v02r01.20200101.nc
# oisst-avhrr-v02r01.20210101.nc
# oisst-avhrr-v02r01.20220101.nc
# oisst-avhrr-v02r01.20230101.nc
# oisst-avhrr-v02r01.20240101.nc
# oisst-avhrr-v02r01.20250101.nc

print("Upload your NOAA OISST NetCDF .nc files below:")
uploaded = files.upload()

nc_files = list(uploaded.keys())
print("Files uploaded:")
for file in nc_files:
    print("-", file)

# =====

DATASET_NAME = "NOAA Optimum Interpolation Sea Surface Temperature (OISST) Version 2.1 High Resolution Dataset"
DATASET_LINK = "https://www.psl.noaa.gov/data/gridded/data.noaa.oisst.v2.highres.html"
PRIMARY_VARIABLE = "anom"
REGION_NAME = "North Atlantic Ocean"
SPECIES_FOCUS = "Atlantic Bluefin Tuna"

print("Dataset:", DATASET_NAME)
print("Source:", DATASET_LINK)
print("Primary variable:", PRIMARY_VARIABLE, "= Sea Surface Temperature Anomaly")
print("Region:", REGION_NAME)
print("Species focus:", SPECIES_FOCUS)

sample_ds = xr.open_dataset(nc_files[0])
print(sample_ds)

print("Available variables:", list(sample_ds.data_vars))
print("Coordinates:", list(sample_ds.coords))

# =====
# 5. DEFINE NORTH ATLANTIC REGION
# NOAA OISST longitude values are usually 0 to 360.
# North Atlantic approximation:
# Latitude: 0 to 60 N
# Longitude: 280 to 360 E and 0 to 20 E
# This captures the western and eastern North Atlantic.

LAT_MIN = 0
LAT_MAX = 60
LON_MIN_WEST = 280

```

```

LON_MAX_WEST = 360
LON_MIN_EAST = 0
LON_MAX_EAST = 20

# =====
# 6. EXTRACT DAILY NORTH ATLANTIC SST ANOMALY
# Each file represents one daily NOAA OISST grid.
# The code extracts the regional mean SST anomaly for each day.

records = []

for nc_file in sorted(nc_files):
    ds = xr.open_dataset(nc_file)

    # Select the anomaly variable.
    # OISST files usually store it as: anom(time, zlev, lat, lon)
    anom = ds["anom"]

    # Remove zlev dimension if present.
    if "zlev" in anom.dims:
        anom = anom.isel(zlev=0)

    # Select latitude range.
    anom = anom.sel(lat=slice(LAT_MIN, LAT_MAX))

    # Select North Atlantic longitudes using two slices because longitude wraps from 0 to 360.
    anom_west = anom.sel(lon=slice(LON_MIN_WEST, LON_MAX_WEST))
    anom_east = anom.sel(lon=slice(LON_MIN_EAST, LON_MAX_EAST))
    anom_region = xr.concat([anom_west, anom_east], dim="lon")

    # Calculate daily regional mean anomaly.
    daily_mean_anom = float(anom_region.mean(dim=["lat", "lon"], skipna=True).values.squeeze())

    # Count valid ocean grid cells used for that daily average.
    valid_grid_cells = int(anom_region.count(dim=["lat", "lon"]).values.squeeze())

    # Extract time.
    date = pd.to_datetime(ds["time"].values[0])

    records.append({
        "date": date,
        "temperature_anomaly": daily_mean_anom,
        "valid_grid_cells": valid_grid_cells,
        "source_file": nc_file
    })

regional_df = pd.DataFrame(records).sort_values("date").reset_index(drop=True)

print("Regional daily dataset created.")
print("Sample size:", len(regional_df), "daily observations")
print("Date range:", regional_df["date"].min(), "to", regional_df["date"].max())
print("Average valid grid cells per day:", round(regional_df["valid_grid_cells"].mean(), 0))
display(regional_df)

# =====
# 7. FEATURE ENGINEERING

# These features match the EDA and preprocessing reports:
# - rolling_temp: 7-day rolling average
# - temp_change: first difference / daily change
# - volatility: 7-day rolling standard deviation
# - ESMS: normalized composite variability score

WINDOW = 7

feature_df = regional_df.copy()

feature_df["rolling_temp"] = feature_df["temperature_anomaly"].rolling(
    window=WINDOW,
    min_periods=1
).mean()

feature_df["temp_change"] = feature_df["temperature_anomaly"].diff().fillna(0)

feature_df["volatility"] = feature_df["temperature_anomaly"].rolling(
    window=WINDOW,
    min_periods=2
).std().fillna(0)

```

```

feature_df["abs_temp_change"] = feature_df["temp_change"].abs()

# Min-max scaling helper

def minmax_scale(series):
    minimum = series.min()
    maximum = series.max()
    denominator = maximum - minimum
    if denominator == 0:
        return pd.Series(np.zeros(len(series)), index=series.index)
    return (series - minimum) / denominator

feature_df["volatility_norm"] = minmax_scale(feature_df["volatility"])
feature_df["abs_temp_change_norm"] = minmax_scale(feature_df["abs_temp_change"])

# Environmental Stress Movement Score
# Higher ESMS = stronger environmental instability signal.
feature_df["ESMS"] = feature_df["volatility_norm"] + feature_df["abs_temp_change_norm"]

print("Feature engineering complete.")
display(feature_df)

# =====
# 8. SUMMARY STATISTICS AND CORRELATION MATRIX

analysis_cols = [
    "temperature_anomaly",
    "rolling_temp",
    "temp_change",
    "volatility",
    "ESMS"
]

summary_stats = feature_df[analysis_cols].describe().round(3)
correlation_matrix = feature_df[analysis_cols].corr().round(3)

print("Summary Statistics")
display(summary_stats)

print("Correlation Matrix")
display(correlation_matrix)

# =====
# 9. FIGURE 1: TEMPERATURE ANOMALIES OVER TIME

plt.figure(figsize=(11, 5))
plt.plot(feature_df["date"], feature_df["temperature_anomaly"], marker="o")
plt.title("Figure 1. North Atlantic SST Anomalies Over Time")
plt.xlabel("Date")
plt.ylabel("Regional Mean SST Anomaly")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("figure_1_temperature_anomalies_over_time.png", dpi=300)
plt.show()

# =====
# 10. FIGURE 2: DISTRIBUTION OF VOLATILITY

plt.figure(figsize=(8, 5))
plt.hist(feature_df["volatility"], bins=10)
plt.title("Figure 2. Distribution of Volatility")
plt.xlabel("Volatility")
plt.ylabel("Frequency")
plt.tight_layout()
plt.savefig("figure_2_volatility_distribution.png", dpi=300)
plt.show()

# =====
# 11. FIGURE 3: DISTRIBUTION OF TEMPERATURE CHANGE

plt.figure(figsize=(8, 5))
plt.hist(feature_df["temp_change"], bins=10)
plt.title("Figure 3. Distribution of Temperature Change")

```

```

plt.xlabel("Temperature Change")
plt.ylabel("Frequency")
plt.tight_layout()
plt.savefig("figure_3_temperature_change_distribution.png", dpi=300)
plt.show()

# =====
# 12. FIGURE 4: TEMPERATURE ANOMALY VS ESMS

plt.figure(figsize=(8, 5))
plt.scatter(feature_df["temperature_anomaly"], feature_df["ESMS"], alpha=0.8)

if len(feature_df) > 1:
    x = feature_df["temperature_anomaly"]
    y = feature_df["ESMS"]
    slope, intercept = np.polyfit(x, y, 1)
    plt.plot(x, slope * x + intercept)

plt.title("Figure 4. Temperature Anomaly vs ESMS")
plt.xlabel("Regional Mean SST Anomaly")
plt.ylabel("Environmental Stress Movement Score")
plt.tight_layout()
plt.savefig("figure_4_temperature_vs_esms.png", dpi=300)
plt.show()

# =====
# 13. FIGURE 5: CORRELATION HEATMAP

plt.figure(figsize=(8, 6))
plt.imshow(correlation_matrix)
plt.colorbar(label="Correlation")
plt.title("Figure 5. Correlation Heatmap")
plt.xticks(
    range(len(correlation_matrix.columns)),
    correlation_matrix.columns,
    rotation=45,
    ha="right"
)
plt.yticks(range(len(correlation_matrix.index)), correlation_matrix.index)

for i in range(len(correlation_matrix.index)):
    for j in range(len(correlation_matrix.columns)):
        plt.text(j, i, correlation_matrix.iloc[i, j], ha="center", va="center")

plt.tight_layout()
plt.savefig("figure_5_correlation_heatmap.png", dpi=300)
plt.show()

# =====
# 14. OPTIONAL ENVIRONMENTAL REGIME GROUPING

# This supports the preprocessing report's future unsupervised modeling idea.
# It groups days into lower, moderate, and higher variability based on ESMS quantiles.

low_cutoff = feature_df["ESMS"].quantile(0.33)
high_cutoff = feature_df["ESMS"].quantile(0.66)

feature_df["environmental_regime"] = np.where(
    feature_df["ESMS"] <= low_cutoff,
    "lower_variability",
    np.where(feature_df["ESMS"] >= high_cutoff, "higher_variability", "moderate_variability")
)

print("Environmental Regime Counts")
display(feature_df["environmental_regime"].value_counts())

plt.figure(figsize=(8, 5))
feature_df["environmental_regime"].value_counts().plot(kind="bar")
plt.title("Figure 6. Environmental Regime Counts Based on ESMS")
plt.xlabel("Environmental Regime")
plt.ylabel("Number of Days")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.savefig("figure_6_environmental_regime_counts.png", dpi=300)
plt.show()

```

```

# =====
# 15. HYPERPARAMETER TUNING AND MODEL EVALUATION
# =====

### In this section, I move from exploratory analysis into modeling.
### I compare a baseline linear model (Ridge Regression) with a more flexible model (Random Forest).

### Because my dataset is small, I use Leave-One-Out Cross Validation instead of a traditional train/test split.
### I tune hyperparameters to improve model performance while also controlling for overfitting.

from sklearn.model_selection import LeaveOneOut, GridSearchCV, cross_val_score
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import pandas as pd
import numpy as np

# -----
# Create lagged features to avoid data leakage.
# I do NOT want to predict temperature anomaly using values
# from the same day that already contain the answer.
# -----

model_df = feature_df.copy()

model_df["lag1_temperature_anomaly"] = model_df["temperature_anomaly"].shift(1)
model_df["lag1_rolling_temp"] = model_df["rolling_temp"].shift(1)
model_df["lag1_temp_change"] = model_df["temp_change"].shift(1)
model_df["lag1_volatility"] = model_df["volatility"].shift(1)
model_df["lag1_ESMS"] = model_df["ESMS"].shift(1)

model_df = model_df.dropna().reset_index(drop=True)

X = model_df[
    [
        "lag1_temperature_anomaly",
        "lag1_rolling_temp",
        "lag1_temp_change",
        "lag1_volatility",
        "lag1_ESMS"
    ]
]

y = model_df["temperature_anomaly"]

print("Modeling dataset shape:", model_df.shape)
display(model_df[["date", "temperature_anomaly"] + list(X.columns)])

# =====
# BASELINE MODEL: RIDGE REGRESSION
# =====

loo = LeaveOneOut()

ridge = Ridge()

ridge_param_grid = {
    "alpha": [0.01, 0.1, 1, 10, 100]
}

ridge_grid = GridSearchCV(
    estimator=ridge,
    param_grid=ridge_param_grid,
    cv=loo,
    scoring="neg_mean_squared_error"
)

ridge_grid.fit(X, y)

best_ridge = ridge_grid.best_estimator_

ridge_mse_scores = -cross_val_score(
    best_ridge,
    X,
    y,

```

```

        cv=loo,
        scoring="neg_mean_squared_error"
    )

    ridge_mae_scores = -cross_val_score(
        best_ridge,
        X,
        y,
        cv=loo,
        scoring="neg_mean_absolute_error"
    )

    print("Best Ridge Parameters:")
    print(ridge_grid.best_params_)

    print("Ridge Mean CV RMSE:", np.sqrt(ridge_mse_scores.mean()))
    print("Ridge Mean CV MAE:", ridge_mae_scores.mean())

    # =====
    # TUNED MODEL: RANDOM FOREST REGRESSOR
    # =====

    rf = RandomForestRegressor(random_state=42)

    rf_param_grid = {
        "n_estimators": [50, 100, 200],
        "max_depth": [2, 3, None],
        "min_samples_split": [2, 3],
        "min_samples_leaf": [1, 2]
    }

    rf_grid = GridSearchCV(
        estimator=rf,
        param_grid=rf_param_grid,
        cv=loo,
        scoring="neg_mean_squared_error",
        n_jobs=-1
    )

    rf_grid.fit(X, y)

    best_rf = rf_grid.best_estimator_

    rf_mse_scores = -cross_val_score(
        best_rf,
        X,
        y,
        cv=loo,
        scoring="neg_mean_squared_error"
    )

    rf_mae_scores = -cross_val_score(
        best_rf,
        X,
        y,
        cv=loo,
        scoring="neg_mean_absolute_error"
    )

    print("Best Random Forest Parameters:")
    print(rf_grid.best_params_)

    print("Random Forest Mean CV RMSE:", np.sqrt(rf_mse_scores.mean()))
    print("Random Forest Mean CV MAE:", rf_mae_scores.mean())

    # =====
    # MODEL COMPARISON TABLE
    # =====

    model_comparison = pd.DataFrame([
        {
            "Model": "Tuned Ridge Regression",
            "Best Parameters": ridge_grid.best_params_,
            "Mean CV RMSE": np.sqrt(ridge_mse_scores.mean()),
            "Mean CV MAE": ridge_mae_scores.mean()
        },
        {
            "Model": "Tuned Random Forest"

```

```

        model : tuned_random_forest ,
        "Best Parameters": rf_grid.best_params_,
        "Mean CV RMSE": np.sqrt(rf_mse_scores.mean()),
        "Mean CV MAE": rf_mae_scores.mean()
    }
])

display(model_comparison.round(4))
model_comparison.to_csv("model_comparison_table.csv", index=False)

# =====
# OVERFITTING CHECK
# =====

best_rf.fit(X, y)

train_predictions = best_rf.predict(X)

overfitting_check = pd.DataFrame({
    "Metric": ["Train RMSE", "Train MAE", "Train R²"],
    "Value": [
        np.sqrt(mean_squared_error(y, train_predictions)),
        mean_absolute_error(y, train_predictions),
        r2_score(y, train_predictions)
    ]
})

display(overfitting_check.round(4))
overfitting_check.to_csv("overfitting_check.csv", index=False)

# =====
# FEATURE IMPORTANCE FROM BEST RANDOM FOREST
# =====

feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": best_rf.feature_importances_
}).sort_values(by="Importance", ascending=False)

display(feature_importance.round(4))
feature_importance.to_csv("feature_importance.csv", index=False)

plt.figure(figsize=(8, 5))
plt.bar(feature_importance["Feature"], feature_importance["Importance"])
plt.title("Figure 7. Feature Importance from Tuned Random Forest")
plt.xlabel("Feature")
plt.ylabel("Importance")
plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.savefig("figure_7_feature_importance.png", dpi=300)
plt.show()

# =====
# 15. SAVE

feature_df.to_csv("processed_noaa_oisst_environmental_stress_data.csv", index=False)
summary_stats.to_csv("summary_statistics.csv")
correlation_matrix.to_csv("correlation_matrix.csv")

print("Saved files:")
print("- processed_noaa_oisst_environmental_stress_data.csv")
print("- summary_statistics.csv")
print("- correlation_matrix.csv")
print("- figure_1_temperature_anomalies_over_time.png")
print("- figure_2_volatility_distribution.png")
print("- figure_3_temperature_change_distribution.png")
print("- figure_4_temperature_vs_esms.png")
print("- figure_5_correlation_heatmap.png")
print("- figure_6_environmental_regime_counts.png")

# =====
# 16. DOWNLOAD OUTPUTS FROM COLAB

output_files = [
    "processed_noaa_oisst_environmental_stress_data.csv",
    "summary_statistics.csv",
    "correlation_matrix.csv",
    "figure 1 temperature anomalies over time.png",

```

```
"figure_2_volatility_distribution.png",  
"figure_3_temperature_change_distribution.png",  
"figure_4_temperature_vs_esms.png",  
"figure_5_correlation_heatmap.png",  
"figure_6_environmental_regime_counts.png"  
"model_comparison_table.csv",  
"overfitting_check.csv",  
"feature_importance.csv",  
"figure_7_feature_importance.png"  
]  
  
# for output_file in output_files:  
#     files.download(output_file)
```







|   |                     |          |       |                                |          |           |          |          |
|---|---------------------|----------|-------|--------------------------------|----------|-----------|----------|----------|
| 2 | 2022-01-01 12:00:00 | 0.732922 | 63862 | oisst-avhrr-v02r01.20220101.nc | 0.594416 | 0.184933  | 0.122102 | 0.184933 |
| 3 | 2023-01-01 12:00:00 | 0.741130 | 63862 | oisst-avhrr-v02r01.20230101.nc | 0.631095 | 0.008207  | 0.123776 | 0.008207 |
| 4 | 2024-01-01 12:00:00 | 0.949585 | 63862 | oisst-avhrr-v02r01.20240101.nc | 0.694793 | 0.208455  | 0.178263 | 0.208455 |
| 5 | 2025-01-01 12:00:00 | 0.931619 | 63862 | oisst-avhrr-v02r01.20250101.nc | 0.734264 | -0.017966 | 0.186467 | 0.017966 |

Summary Statistics

|       | temperature_anomaly | rolling_temp | temp_change | volatility | ESMS  |  |
|-------|---------------------|--------------|-------------|------------|-------|--|
| count | 6.000               | 6.000        | 6.000       | 6.000      | 6.000 |  |
| mean  | 0.734               | 0.614        | 0.072       | 0.107      | 0.947 |  |
| std   | 0.186               | 0.092        | 0.099       | 0.076      | 0.729 |  |
| min   | 0.502               | 0.502        | -0.018      | 0.000      | 0.000 |  |
| 25%   | 0.594               | 0.542        | 0.002       | 0.055      | 0.470 |  |
| 50%   | 0.737               | 0.613        | 0.027       | 0.123      | 0.895 |  |
| 75%   | 0.884               | 0.679        | 0.150       | 0.165      | 1.428 |  |
| max   | 0.950               | 0.734        | 0.208       | 0.186      | 1.956 |  |

Correlation Matrix

|                     | temperature_anomaly | rolling_temp | temp_change | volatility | ESMS  |  |
|---------------------|---------------------|--------------|-------------|------------|-------|--|
| temperature_anomaly | 1.000               | 0.979        | 0.352       | 0.985      | 0.832 |  |
| rolling_temp        | 0.979               | 1.000        | 0.160       | 0.972      | 0.711 |  |
| temp_change         | 0.352               | 0.160        | 1.000       | 0.341      | 0.803 |  |
| volatility          | 0.985               | 0.972        | 0.341       | 1.000      | 0.832 |  |
| ESMS                | 0.832               | 0.711        | 0.803       | 0.832      | 1.000 |  |

Figure 1. North Atlantic SST Anomalies Over Time

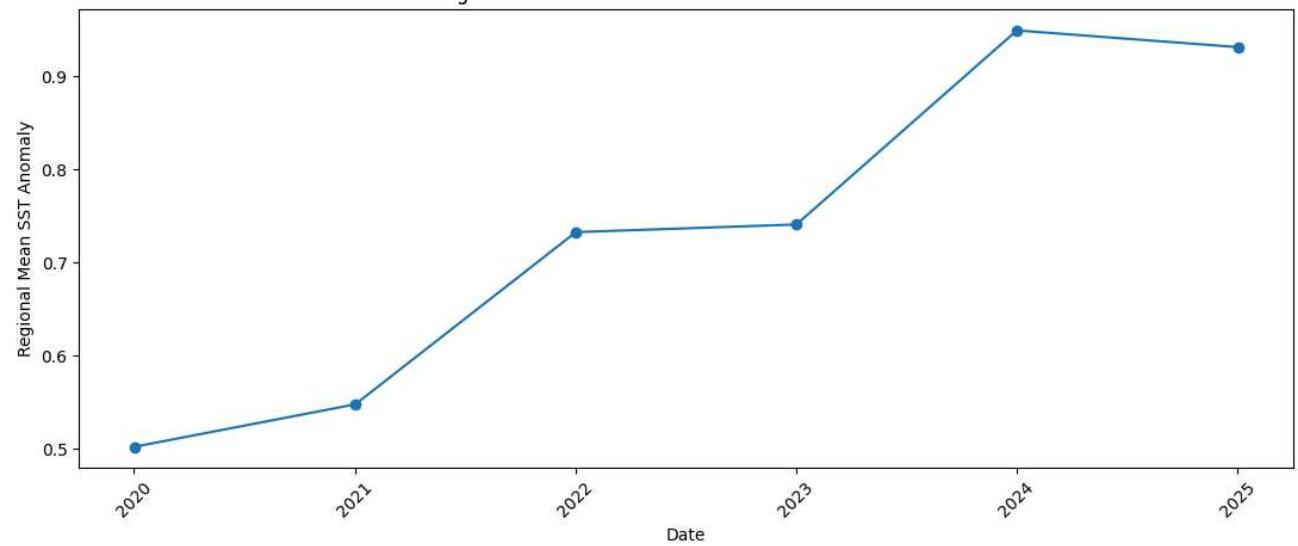
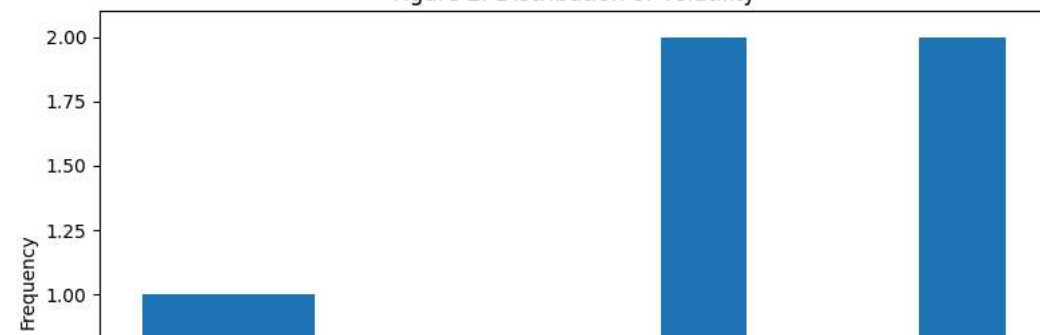


Figure 2. Distribution of Volatility



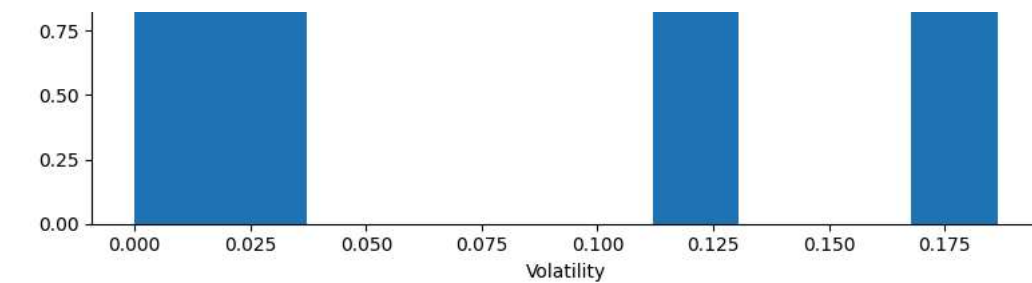


Figure 3. Distribution of Temperature Change

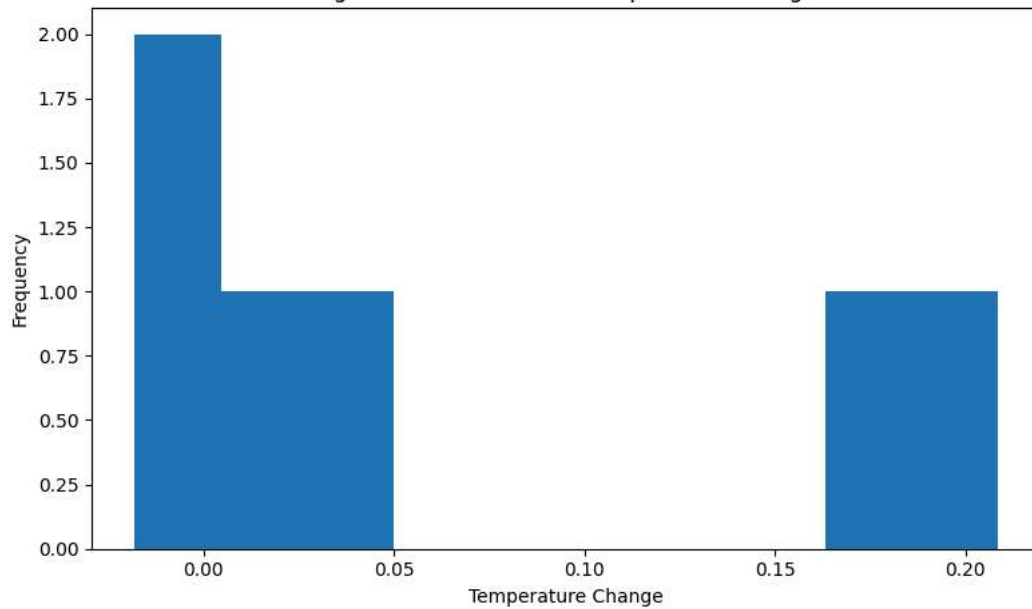


Figure 4. Temperature Anomaly vs ESMS

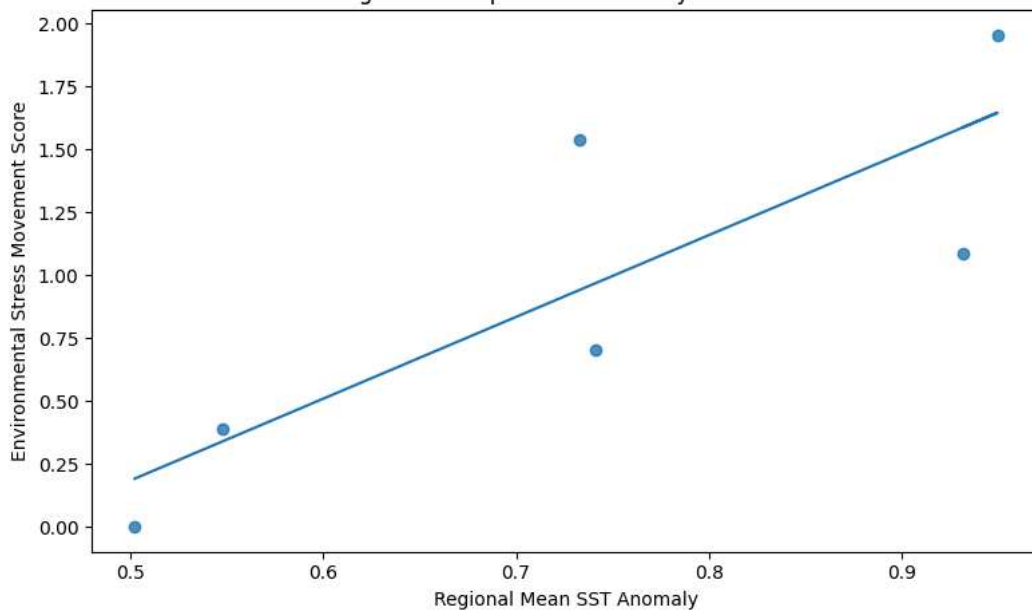


Figure 5. Correlation Heatmap

